

Personalized Hotel Ranking for Expedia Search Results

Group 111¹, Anushka Sen [2873501]¹, Akshit Anchan [2878965]¹, and Prita Suresh [2884313]¹

Vrije Universiteit Amsterdam, Data Mining Techniques 2025/2026

1 Introduction

When a user searches for a hotel on Expedia, the platform returns a list of candidate properties. The order in which these properties appear matters: hotels shown near the top receive more attention, and a better ranking can increase bookings. From a business perspective, the goal is to place attractive and relevant hotels near the top of each search result page while still adapting to the user’s destination, budget, and travel context. The task is to predict which hotel a user is most likely to book, given features about the search query, the candidate hotels, and their competitive pricing. Predictions are evaluated using NDCG@5 [1], a ranking metric that rewards placing the booked hotel within the top five results.

Hotel relevance is relative to the other candidates in the same search and to the user’s apparent travel context, so we treat this as a learning-to-rank problem over complete search result lists rather than row-level classification. Our submitted system combines boosted-tree rankers with engineered search, hotel, pricing, and collaborative-filtering features. The report follows the CRISP-DM process model from data understanding through deployment.

2 Related Work

The dataset originates from the ICDM 2013 “Personalize Expedia Hotel Searches” Kaggle competition [2]. We used only the VU in-class dataset for training and prediction, but several published approaches to the original competition informed our methodology.

Liu et al. [3] described a top-performing approach that combined diverse ranking models with extensive feature engineering. Their most predictive signals were within-search price ranks, hotel-level click and booking aggregates, competitor rate differentials, and position-corrected quality estimates. They used a LambdaMART ensemble over these features. The key takeaway for our work was that feature design mattered more than model selection: even simple pointwise models performed well when the input features captured relative position within a search.

Users are more likely to click results shown near the top, even when those results are not the most relevant, a position bias problem that has been studied extensively in search ranking. Joachims et al. [5] demonstrated this effect in web

search and proposed randomized interventions to correct for it. The Expedia dataset includes a `random_bool` field that marks searches where display order was randomized, the same intervention Joachims advocated. We use this subset to compute position-debiased click and booking rates, rather than treating clicks from normal ranked lists as direct measures of hotel quality.

Matrix factorization for collaborative filtering is a standard technique in recommender systems [6]. In our setting, destinations and hotels form the two axes of an interaction matrix, and latent factor decomposition captures patterns of destination-hotel affinity that are not visible in the raw features.

3 Data Understanding

3.1 Dataset Overview

The training set contains 4,958,347 rows, each representing one hotel shown in response to one search query. There are 199,795 unique searches and 129,113 unique hotels across 34 Expedia points of sale, 18,127 destinations, and 172 hotel countries. The test set has a similar scale. Each search returns roughly 25 candidate hotels on average, and each row carries 54 raw columns before feature engineering.

The booking rate is low: only 2.79% of candidate-hotel rows correspond to a booking, and all bookings are also clicks. The conditional probability of booking given a click is 62.4%. Row-level accuracy is not informative for ranking quality, which is why the evaluation uses search-level NDCG@5. Booking rates also vary by star rating, peaking at 3.3% for four-star hotels and dipping for both zero/one-star and five-star tiers, suggesting price-quality interaction matters more than star rating alone.

3.2 Missing Values

Missing values are concentrated in two areas. The eight sets of competitor columns (`comp1_rate` through `comp8_rate_percent_diff`) are roughly 52 to 98% null, because competitive pricing data is only available for a subset of hotels. Visitor history fields (`visitor_hist_starrating` and `visitor_hist_adr_usd`) are 94.9% null because most users have no prior purchase history on the platform. The field `orig_destination_distance` is 32.4% null, and `prop_location_score2` is 22.0% null.

An important observation is that missingness is informative. The booking rate for users who have purchase history is 3.61%, versus 2.75% for users who do not. Hotels with any competitor pricing data have a higher booking rate (2.91%) than those without (2.56%). Rather than imputing these fields and throwing away the missingness signal, we retain binary indicator flags.

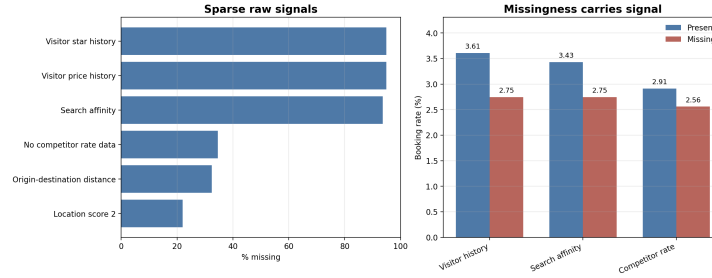


Fig. 1. Missingness in key fields and booking-rate differences when selected signals are present versus missing. Null values carry predictive signal, especially for visitor history, search affinity, and competitor pricing.

3.3 Position Bias

The training data includes a `position` field indicating where each hotel appeared in the search results. The test set does not include position. The `random_bool` field marks rows where the display order was randomized, which account for 29.6% of training rows.

Under normal display ordering, the hotel at position 1 receives a click rate of 21.5%, nearly five times the overall average of 4.47%. Under random ordering, position 1 still gets a 14.0% click rate, confirming that position bias in user attention is substantial even when hotel quality is uncorrelated with display order. The gap between 21.5% (normal) and 14.0% (random) reflects the quality-sorting effect that normal ordering adds on top of positional attention. The booking rate under random ordering is 0.53%, much lower than the 3.74% under normal ordering, because the normal order tends to place relevant hotels higher. The implication for feature engineering is direct: any hotel-level statistic computed from training data (such as historical click rate) can be confounded by the position at which the hotel was displayed. To reduce this confounding, we compute hotel-level aggregates exclusively from the `random_bool=1` subset.

4 Data Preparation and Feature Engineering

Starting from the raw 54-column dataset, we engineered 273 features organized into six families. All features are computed at the search-hotel row level.

Before feature construction, we made three preparation decisions. First, all continuous columns are loaded as `float32` and categoricals as `int8/int16` to keep memory under control on a 5-million-row dataset. Second, for missing values we chose flag-and-keep over blind imputation: binary indicator columns preserve the informative missingness identified in the EDA. The one exception is `prop_location_score2`, where we compute missingness and pre-imputation relative features first, then impute with the Q1 value per `prop_country_id` fitted on training data only. Third, the validation split uses `GroupShuffleSplit` by

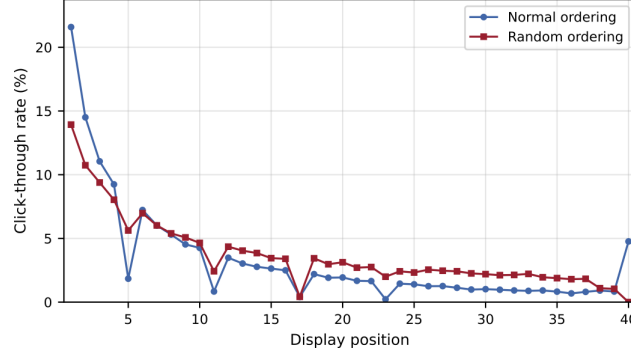


Fig. 2. Click-through rate by display position under normal and randomized ordering. The random-ordering curve isolates position bias. The gap between the two curves reflects the additional quality-sorting effect under normal ordering.

`srch_id` so that all hotels from a single search appear in the same fold. Target-encoded priors use 5-fold **GroupKFold** out-of-fold encoding for training rows and full-train mappings for validation and test rows. Other label-derived lookup features are fitted only from the training split, often using the randomized subset, and then mapped forward.

4.1 Within-Search Relative Features

A \$200 hotel that is the cheapest option in its search means something very different from a \$200 hotel that is the most expensive. To capture these within-search comparisons, for each of five core variables (`price_usd`, `prop_starrating`, `prop_review_score`, `prop_location_score1`, `prop_location_score2`) we compute within-search percentile rank, deviation from search mean, deviation from search median, search minimum, maximum, and standard deviation. We add the total count of hotels in the search, binary flags for cheapest and most expensive hotel, and search-level context features: mean price, price coefficient of variation, brand fraction, and mean star rating. These features capture the competitive position of each hotel within its choice set.

4.2 Position-Debiased Hotel Quality

Using only `random_bool=1` training rows, we compute Bayesian-smoothed hotel-level click-through and booking rates:

$$\hat{r}_{\text{hotel}} = \frac{\text{count}_{\text{hotel}} \cdot r_{\text{hotel}} + m \cdot r_{\text{global}}}{\text{count}_{\text{hotel}} + m} \quad (1)$$

where $\text{count}_{\text{hotel}}$ is randomized-row support, r_{hotel} is the observed hotel rate, $m = 30$ is the smoothing parameter, and r_{global} is the global rate. The resulting

estimates are less confounded by position because the randomized subset breaks the link between hotel quality and displayed rank. The same approach is applied at four levels of granularity: per hotel, per destination, per destination-hotel pair, and per visitor-country-to-hotel-country pair.

We extend these priors to finer-grained keys. Destination-by-star-rating priors capture booking rate, click rate, and click-only rate per destination and star tier. Hotel-by-booking-window priors split hotels into eight booking-window buckets. Both use Bayesian smoothing with $m = 50$. A budget hotel may perform well for last-minute domestic travel but poorly for advance international bookings, and these priors let the model distinguish those cases.

4.3 Collaborative Filtering via SVD

We use truncated SVD as a collaborative filtering technique [6]. From the randomized (`random_bool=1`) training rows, we build a destination-hotel interaction matrix where each entry is weighted by relevance ($5 \times \text{booking_bool} + 1 \times \text{click_only}$). The matrix is decomposed into 20 latent factors per destination and 20 per hotel using `TruncatedSVD` from scikit-learn.

The decomposition yields 40 latent factor features plus a dot-product affinity score (`svd_affinity_score`) and a cosine similarity variant. Hotels and destinations that frequently co-occur in bookings get similar latent representations even if they share no explicit attributes. As a standalone ranker, the SVD affinity score is modest, outperforming a simple ranking by `prop_location_score1` but well below the tree models (see Table 2). The SVD features took significant engineering effort for that standalone score, but their value became clear once we fed the latent factors into the tree models, where an SVD-derived affinity rank feature appears among the top-20 features by gain (Figure 3).

We also compute item-based collaborative filtering rates by grouping hotels by site, by visitor country, and by destination, each with Bayesian smoothing ($m = 30$). In the final model, we add contextual collaborative filtering: separate SVD-based affinity scores conditioned on site (`site_prop_cf_score`) and on the visitor-destination pair (`visitor_dest_prop_cf_score`).

4.4 Target-Encoded Priors with Out-of-Fold Encoding

Hotel popularity is a strong booking predictor, but encoding it naively from training labels creates target leakage. We use 5-fold `GroupKFold` by `srch_id` to compute out-of-fold (OOF) booking, click, and click-only rates per `prop_id`. For each fold, the rates are computed from the other four folds and assigned to the held-out fold. Validation and test rows receive rates from the full training set. All rates use Bayesian smoothing with $m = 50$.

Property-level price aggregates (`prop_id_price_mean`, `prop_id_price_std`), computed from training data, and the deviation of the current row’s price from its property mean capture whether a hotel is being offered at an unusual price compared to its historical average.

In the final model, hierarchical priors at three key combinations (hotel-by-destination, hotel-by-visitor-country, and hotel-by-booking-window-bucket) use Bayesian backoff to the parent hotel-only prior, producing a booking rate estimate, a lift-over-parent ratio, and a reliability count.

4.5 Competitor and Value Features

The competitor columns are roughly 52 to 98% null, which limits their direct usefulness. We aggregate the non-null entries into seven features: count of competitors with lower prices, count with higher prices, count with the same price, count with no inventory, total competitors with data, mean price difference, and maximum price difference. A composite `comp_advantage_score` and a `comp_cheaper_frac` summarize competitive position.

Pareto dominance features capture whether a hotel is both cheaper and higher-quality than all alternatives in the search: `is_price_quality_pareto` (binary flag), `quality_rank_minus_price_rank` (signed gap), and gap-to-best features for five axes.

4.6 Contextual and Market Features

A separate LightGBM model predicts the hotel’s display position using only features available at test time (excluding `position`, `click_bool`, `booking_bool`, and all target-derived features). The predicted score (`expedia_sort_score`) and its within-search rank provide an estimate of Expedia’s proprietary quality signal that is otherwise lost at test time. Derived features include the gap to the best predicted sort score and the discrepancy between predicted sort rank and quality rank.

Displayed prices follow market-specific conventions. We compute market-level median prices per night and residuals from those medians to capture whether a hotel is expensive or cheap relative to its market context, alongside temporal features: check-in month, check-in day-of-week, and booking-window bucket.

5 Modeling

5.1 Techniques

We use three distinct technique families: LightGBM with multiple objectives, CatBoost YetiRank, and SVD collaborative filtering. The last satisfies the assignment requirement for a recommender systems technique. Relevance is graded as 5 for a booking, 1 for a click without a booking, and 0 otherwise.

LightGBM with Multiple Ranking Objectives. LightGBM [7] is a gradient boosted tree framework. We train five LightGBM variants spanning pointwise classification, LambdaRank, and RankXENDCG objectives:

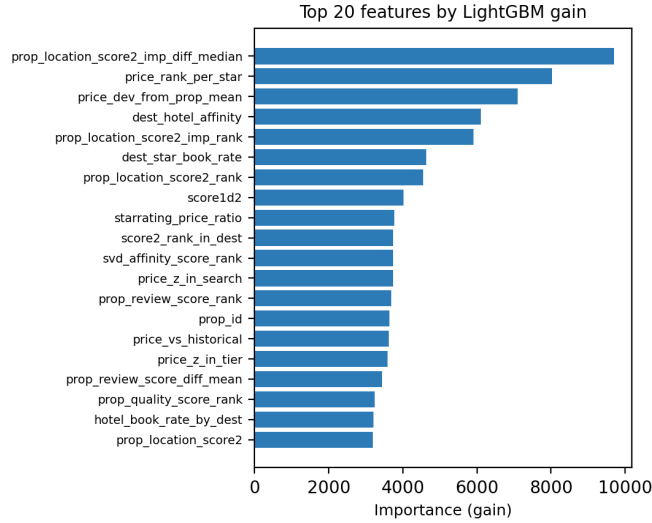


Fig. 3. Top 20 features by LightGBM gain on a 15% sample. Location score, price-related features, and target-encoded priors dominate. An SVD-derived affinity rank feature appears in the top 20 despite SVD being weak as a standalone ranker.

1. **Pointwise classification.** We train two pointwise binary models, one with `booking_bool` as target and one with `click_bool`. Click propensity differs from booking propensity because many clicks do not lead to bookings, so the two are kept as separate models. The click model uses 255 leaves and learning rate 0.025. The booking model uses the shared hyperparameters in Table 1.
2. **LambdaRank, including a truncation@5 variant.** The objective is `lambdarank` [4], which uses pairwise gradients weighted by the NDCG change induced by ranking swaps. It compares pairs of hotels within each search and adjusts the model to rank booked hotels above non-booked ones. We train two variants: the default LambdaRank and a truncation@5 variant (`lambdarank.truncation_level=5`) that focuses gradient updates on the top five positions, aligning training with the NDCG@5 metric.
3. **RankXENDCG.** The objective is `rank_xendcg` [9], which uses a differentiable approximation to expected NDCG. We train five independent RankX-ENDCG models with different random seeds (42, 2026, 777, 1337, 9001) and average their per-search z-normalized predictions. Averaging across seeds reduces variance without adding model complexity. Each seed also applies query-local negative dropout, randomly dropping 35% of non-booked candidates per search during training, which creates genuinely different training distributions and makes the five models more diverse.

Shared LightGBM hyperparameters appear in Table 1. The linear gain policy was chosen to be consistent with the assignment’s NDCG@5 definition, where $DCG = \sum rel_i / \log_2(rank_i + 1)$ uses linear relevance gains. An ablation with the default exponential gain scored lower on validation.

CatBoost YetiRank. CatBoost [8] differs from LightGBM mainly in its native handling of categorical features and its symmetric (balanced) tree structure. We use the YetiRank loss, which optimizes a pairwise ranking objective. CatBoost handles `prop_id`, `srch_destination_id`, `site_id`, and `visitor_location_country_id` as categoricals directly, without one-hot encoding or target encoding. Training used depth 8, learning rate 0.03, L2 leaf regularization 3.0, and up to 9,000 iterations with validation NDCG@5 early stopping at 400 rounds.

CatBoost was weaker standalone at 0.4052 but still earned 8.4% blend weight because its symmetric-tree construction and different ranking loss produce partly uncorrelated errors compared to the LightGBM models.

SVD Collaborative Filtering. The SVD collaborative filtering described in Section 4.3 is weak as a standalone ranker compared to the tree models. Its primary value is as input features: the 40 latent factors and affinity-derived features give the tree models additional destination-hotel interaction signals.

5.2 Ensemble

The base model scores are combined using optimized signed z-score blending. For each model, raw predictions are z-normalized within each search (subtract search mean, divide by search standard deviation). The final score is a weighted sum of z-normalized scores, with weights optimized by Nelder-Mead minimization on validation NDCG@5. The signed variant allows negative weights (penalizing a model that anti-correlates with relevance) and applies L1 normalization with a small L2 penalty ($\lambda = 10^{-4}$).

The optimized weights, in descending order, are: RankXENDCG 25.0%, Pointwise 22.4%, LambdaRank@5 22.1%, LambdaRank 17.1%, CatBoost 8.4%, Click 5.0%. RankXENDCG receives the highest weight, consistent with its position as the strongest base model.

We also tested RRF [10] and a 10-fold OOF meta-stacker over the base scores plus 12 routing features. The meta-stacker had the highest mean validation NDCG@5 (Table 2) but a fold standard deviation of 0.0046 with a low fold of 0.4124. Although the +0.0016 gain was attractive, the instability convinced us it was unreliable, and we selected the signed z-score blend (0.4197) for its stability.

5.3 What Did Not Work

Several ideas were tried and discarded during development.

Table 1. Hyperparameters for all base models.

Parameter	PW	Click	LR	LR@5	XENDCG	CatBoost
Leaves / Depth	511	255	511	511	511	8
Learning rate	0.01	0.025	0.01	0.01	0.01	0.03
L1 / L2 reg.	0.1/1.0	0.05/1.0	0.1/1.0	0.1/1.0	0.1/1.0	-/3.0
Feature frac.	0.8	0.75	0.8	0.8	0.8	-
Max rounds	9000	6000	10000	10000	8000	9000
Best iteration	2843	1041	1032	1092	var.	1588

Segment-specific blend weights. Domestic/international-specific blend weights (0.30 segment-specific and 0.70 global mixing) gained only +0.00020 NDCG@5, below our +0.00025 promotion threshold, and the two weight profiles were nearly identical.

Top-K reranker. A second-stage reranker over the union of each base model’s top-15 candidates scored only 0.245 NDCG@5 because items outside the union received fallback scores that degraded the overall ranking.

An earlier ensemble of two LightGBM models with rank correlation ~ 0.97 showed negligible improvement over either model alone, which motivated the shift toward CatBoost and the alternative ranking objectives described in Section 5.1.

5.4 Hyperparameters

Across the boosted-tree models, we used high-capacity trees, low learning rates, and long schedules stopped at the best validation iteration because the wide, sparse feature space made rare hotel and destination priors easy to overfit. The click model is smaller because click labels are noisier than bookings, while CatBoost uses depth 8 and stronger L2 regularization to control categorical splits. Table 1 summarizes the hyperparameters for all base models.

6 Final Submitted Model

For the final submission, all six models are retrained on the full training set (4,958,347 rows) using the best iteration counts recorded during validation (Table 1). No early stopping is applied on the test set. The six models are the pointwise booking classifier, the click classifier, LambdaRank, LambdaRank with truncation at 5, the 5-seed RankXENDCG average, and CatBoost YetiRank. Together, they use 273 features to produce predictions.

The feature pipeline runs in a fixed order, and this order matters because later stages depend on outputs from earlier ones:

1. Within-search relative features, search context statistics, and price/competitor aggregates.

2. Missing-value indicator flags.
3. Temporal features (check-in month, day of week, booking-window bucket) and market price residuals.
4. SVD collaborative-filtering latent factors, affinity scores, and item-based CF rates from the `random_bool=1` subset.
5. Position-debiased hotel quality estimates, also from the `random_bool=1` subset.
6. OOF target-encoded priors at the property, destination-star, hotel-window, and hierarchical prior levels.
7. Choice-set dominance features (Pareto flags, quality-minus-price rank gaps).
8. Sort emulator score and contextual CF scores.

For test rows, all target-derived lookup maps (steps 4 through 8) are computed from the full training set rather than fold subsets.

Each model’s raw predictions are z-normalized within each search. The z-scores are combined with the signed blend weights from Section 5.2: RankX-ENDCG 25.1%, Pointwise 22.4%, LambdaRank@5 22.1%, LambdaRank 17.1%, CatBoost 8.4%, Click 5.0%. We chose this blend over the meta-stacker because the meta-stacker’s fold standard deviation of 0.0046 made it too unstable to trust on unseen data. Hotels within each search are ranked by descending blended score. The submission file contains 4,959,183 rows.

The model is strongest on domestic searches to popular destinations, where the target-encoded priors draw on thousands of training observations (NDCG@5 0.428). It is weakest on international searches to rare destinations. For those queries, the hotel-destination priors often have fewer than 10 observations and collapse to near-global rates, dropping NDCG@5 to 0.407. Hotels that never appeared in training are a related failure mode: they have no SVD embedding, no target-encoded prior, and no position-debiased quality estimate, so ranking falls back to raw location scores, price, and competitor features alone.

7 Evaluation

7.1 Validation Strategy

The training data is split into train (80%) and validation (20%) subsets using `GroupShuffleSplit` with `srch_id` as the group key and random state 42, producing 3,966,682 training rows and 991,665 validation rows. Splitting by search ensures that all candidate hotels for a given search appear in the same subset, preventing information leakage across the choice set.

All validation metrics are computed after sorting hotels within each `srch_id`, using the same relevance coding as Kaggle (5 for bookings, 1 for click-only, 0 otherwise). We treated validation as the decision source, with public leaderboard scores used only as an external check. The gates were conservative. The domestic/international segment blend gained only +0.000196 over the global signed blend, below the +0.00025 promotion threshold. The top-K reranker was rejected at 0.2453 NDCG@5 despite 0.921 positive recall, and the meta-stacker

Table 2. Validation NDCG@5 for base models and ensembles.

Model	NDCG@5
SVD standalone	0.2196
CatBoost YetiRank	0.4052
Click (LightGBM binary)	0.4112
LambdaRank@5	0.4128
LambdaRank	0.4143
Pointwise (LightGBM binary)	0.4162
RankXENDCG (5-seed avg.)	0.4196
RRF ensemble	0.4187
Signed z-score blend	0.4197
Meta-stacker (diagnostic, not submitted)	0.4213

Table 3. Score progression across development stages.

Stage	Val. NDCG@5	Kaggle
Baseline pointwise/LambdaRank blend	0.4003	0.4009
Corrected features and tuning	0.4026	0.4047
Diverse rankers: RankXENDCG, CatBoost	0.4063	0.4080
OOF meta-ranker and stronger target priors	0.4101	0.4111
Property priors, dominance features, signed z-score blend	0.4149	0.4158
Seed averaging and contextual feature block	0.4165	0.4170
Hierarchical priors, sort emulation, price residuals	0.4197	0.4206

stayed diagnostic because its 0.4213 average came with fold standard deviation 0.0046 and a low fold of 0.4124.

7.2 Base Model Performance

Table 2 reports validation NDCG@5 for each base model and the ensemble.

RankXENDCG is the strongest individual model at 0.4196, followed by the pointwise model at 0.4162. CatBoost is the weakest at 0.4052 but still receives blend weight because it makes partly different ranking errors from the LightGBM models. The signed z-score blend (0.4197) was selected over the meta-stacker (0.4213) because the blend is more robust across folds, as discussed in Section 5.2.

7.3 Iterative Improvement

Table 3 shows how performance improved across development stages.

The Kaggle scores in Table 3 are public-leaderboard values, computed on approximately 50% of the test set. After the competition closed, the private leaderboard (remaining 50%) gave a final score of **0.4221**. This is consistent

Table 4. NDCG@5 and Recall@5 by user segment for the signed z-score blend.

Segment	Condition	NDCG@5	Recall@5
Domestic	visitor_country = prop_country	0.4279	0.589
International	visitor_country $\neq$ prop_country	0.4069	0.562
Family	srch_children_count > 0	0.4363	0.599
Non-family	srch_children_count = 0	0.4149	0.573

with our validation NDCG@5 of 0.4197, supporting the signed z-score blend’s generalization and our decision to reject the less stable meta-stacker.

8 Deployment and Bias Investigation

8.1 Bias Detection

Two deployment risks matter for this task. The first is position bias. Figure 2 shows that hotels near the top of the list receive higher click-through rates even under randomized ordering. If historical click and booking rates are computed directly from normal display logs, a hotel can look strong partly because it was previously placed high.

The second risk is unequal ranking quality across user segments. Table 4 reports NDCG@5 and Recall@5 on the held-out validation predictions. Recall@5 is the fraction of positive-relevance hotel rows placed in the top five.

The domestic/international gap is 0.021 NDCG@5 and 0.027 Recall@5. The family/non-family gap is 0.021 NDCG@5 and 0.026 Recall@5. To quantify this disparity, we compute an equalized performance ratio (EPR), defined as the minimum segment NDCG@5 divided by the maximum:

$$\text{EPR} = \frac{\min(\text{NDCG}_{\text{seg}})}{\max(\text{NDCG}_{\text{seg}})}$$

For the domestic/international split, $\text{EPR} = 0.4069/0.4279 = 0.951$. For the family/non-family split, $\text{EPR} = 0.4149/0.4363 = 0.951$.

The international gap appears to be a data sparsity problem. In the raw data, the median visitor-country/property-country pair has 43 rows for international pairs versus 214 rows for domestic pairs, and randomized rows are fewer for international candidates (580,870 versus 886,307). The position-debiased quality features are therefore noisier for cross-border searches.

8.2 Bias Mitigation

We computed hotel-quality priors from the randomized-order subset (`random_bool=1`) and smoothed them for sparse groups. The feature block contains hotel-level click and booking rates, destination-hotel affinity, destination-star priors, and hotel-by-booking-window priors. The same model also receives

Table 5. Effect of international query upweighting on overall and per-segment NDCG@5, relative to the baseline single-seed RankXENDCG (0.4182).

Variant	Overall	Domestic	International	EPR
Baseline (no upweight, no boost)	0.4182	0.4253	0.4060	0.9545
+ In-process query weight 1.7 (intl.)	0.4167	0.4240	0.4044	0.9539

all-impression OOF property priors. That pairing matters. The all-impression priors preserve strong popularity information, while the randomized-order priors give the model a quality signal less tied to Expedia’s historical production rank.

Segment-level disparity is different from position bias, so we tested it separately. For in-processing, we retrained a representative RankXENDCG model with query weight 1.7 for international searches and 1.0 for domestic searches. For post-processing, we added a constant +0.02 score boost to international-hotel predictions before within-search ranking.

8.3 Mitigation Effectiveness

The randomized-order priors remained useful in the final model. Related features such as destination-hotel affinity and destination-star booking rate appear among the high-gain predictors in Figure 3. This matters because the model can fall back on a quality signal that is less tied to raw historical popularity when the two signals disagree.

Post-processing. The +0.02 boost left the final signed blend unchanged: overall NDCG@5 stayed at 0.420158 and overall Recall@5 stayed at 0.578933. It can change rankings only in searches that mix domestic and international candidates, but only 68 of 39,959 validation searches (0.17%) contain such a mix. In nearly every search, the additive boost applies to all candidates equally and leaves the relative ordering unchanged.

In-processing. Query upweighting reduced performance. Overall NDCG@5 dropped from 0.4182 to 0.4167, international NDCG@5 dropped from 0.4060 to 0.4044, and domestic NDCG@5 dropped from 0.4253 to 0.4240. The EPR was essentially unchanged at 0.9539. Upweighting shifted capacity toward rows where the priors and SVD signals were weaker, reducing accuracy without closing the segment gap.

These results point back to the data. Cross-border queries have weaker priors, weaker SVD evidence, and less randomized support. The tested score calibration and query weighting do not add information. A production version would need richer cross-border signals, such as external hotel quality ratings, travel-pattern features, or deliberately oversampled randomized impressions in sparse markets.

8.4 Scalable Deployment

Deploying this in a production system would involve several considerations.

Architecture. The within-search relative features require all candidate hotels to be loaded before feature computation, which means a two-pass architecture: first retrieve candidates, then compute features and score. SVD factors, target-encoded priors, and market medians are pre-computed lookup tables with $O(1)$ retrieval. For a typical search returning 25 hotels, feature computation and the model inferences would take tens of milliseconds.

Cold start. New hotels lack booking history, SVD embeddings, and target-encoded priors, so the Bayesian smoothing falls back to global rates. A production system would bootstrap new hotels via star rating and location features, updating priors rapidly after the first few hundred impressions.

Retraining and monitoring. The SVD interaction matrix and target-encoded priors should be refreshed daily or weekly. Model weights could be retrained less frequently (monthly), since the ranking objectives are relatively stable. NDCG@5 should be monitored per market segment, and a sustained drop in international search performance could indicate distribution shift. The `random.bool` mechanism in the dataset is essentially a live A/B test, and a production system would use the same approach to evaluate ranking changes before full deployment.

9 What We Learned

Most of what we learned came down to one thing: in a ranking problem, the features matter more than the model. Within-search relative features (price rank, gap to the cheapest option, star rating deviation from the search mean) drove far more improvement than switching between LightGBM objectives. We also had to internalize the discipline of leakage-safe encoding. Target-encoded priors, SVD matrices, and the sort emulator all had to be isolated by fold to keep validation honest. Treating this as a standard classification task would have introduced target leakage and over-optimistic validation estimates.

Iteration speed was a practical constraint. Full-data runs on roughly 5 million rows took over an hour and could hit memory pressure on laptop hardware, so the pipeline had to be disciplined about compact dtypes, cached artifacts, Parquet prediction files, LightGBM `Dataset` objects, and explicit cleanup after large joins or training runs. Validation needed the same skepticism. The meta-stacker scored highest on average, but its fold standard deviation was 0.0046 with one fold at 0.4124, so we trusted stability checks over the point estimate. We would diversify models earlier and spend less time tuning a single LightGBM setup.

Two results caught us off guard. CatBoost scored only 0.4052 standalone, well below the LightGBM models, but it earned 8.4% blend weight because its errors were partly uncorrelated with the others. The other surprise was the bias experiment: upweighting international queries during training actually made international NDCG@5 worse, not better. The performance gap turned out to be a data sparsity problem that no objective weighting could fix. That finding has practical value for deployment, because it points to where to invest (more randomized impressions in sparse markets) rather than where to tune.

References

1. Järvelin, K., Kekäläinen, J.: Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.* **20**(4), 422-446 (2002). <https://doi.org/10.1145/582415.582418>
2. Personalize Expedia Hotel Searches – ICDM 2013. Kaggle (2013). <https://www.kaggle.com/c/expedia-personalized-sort>. Last accessed 15 May 2026
3. Liu, Q., et al.: Combination of diverse ranking models for personalized Expedia hotel searches. arXiv preprint arXiv:1311.7679 (2013). <https://arxiv.org/abs/1311.7679>
4. Burges, C.J.C.: From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research (2010)
5. Joachims, T., Granka, L., Pan, B., Hembrooke, H., Gay, G.: Accurately interpreting clickthrough data as implicit feedback. In: *Proc. 28th ACM SIGIR*, pp. 154-161. ACM (2005). <https://doi.org/10.1145/1076034.1076063>
6. Koren, Y., Bell, R., Volinsky, C.: Matrix factorization techniques for recommender systems. *Computer* **42**(8), 30-37 (2009). <https://doi.org/10.1109/MC.2009.263>
7. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: LightGBM: A highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems* 30, pp. 3146-3154 (2017)
8. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V., Gulin, A.: CatBoost: Unbiased boosting with categorical features. In: *Advances in Neural Information Processing Systems* 31, pp. 6638-6648 (2018)
9. Bruch, S., Han, S., Bendersky, M., Najork, M.: A stochastic treatment of learning to rank scoring functions. In: *Proc. 13th ACM WSDM*, pp. 61-69. ACM (2020). <https://doi.org/10.1145/3336191.3371844>
10. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: *Proc. 32nd ACM SIGIR*, pp. 758-759. ACM (2009). <https://doi.org/10.1145/1571941.1572114>